Homework 4

11_apis.qmd

On Your Own - OMDB

1. Build a data set for a collection of movies by completing the **FILL IN** sections in the code below:

```
omdb_api_key <- Sys.getenv("OMDB_KEY")


# Create a vector of 5 movie names you want to study
#  - must figure out pattern in URL for obtaining different movies
movies <- c("little+women", "atonement", "the+conjuring", "up", "conclave")

# Set up empty tibble
omdb <- tibble(Title = character(), Rated = character(), Genre = character(),
       Actors = character(), Metascore = double(), imdbRating = double(),
       BoxOffice = double())

# Use for loop to run through API request process 5 times,
#   each time filling the next row in the tibble
#  - can do max of 1000 GETs per day
for(i in 1:5) {
  url <- str_c("http://www.omdbapi.com/?t=", movies[i], "&apikey=", omdb_api_key)
  Sys.sleep(0.5)
  onemovie <- GET(url)
  details <- content(onemovie, "parse")
  omdb[i,1] <- details$Title
  omdb[i,2] <- details$Rated  # rating --> where does it live?
  omdb[i,3] <- details$Genre  # genres (single string - could be more than one)
  omdb[i,4] <- details$Actors  # actors (single string - could be more than one)
```

```
  omdb[i,5] <- as.numeric(details$Metascore)  # metascore (be sure it is numeric)
  omdb[i,6] <- as.numeric(details$imdbRating)  # imdb rating (be sure it is numeric)
  omdb[i,7] <- parse_number(str_replace(details$BoxOffice, "^\\$", "")) # box office (be sure
}

omdb
```

```
# A tibble: 5 x 7
  Title         Rated Genre                Actors Metascore imdbRating BoxOffice
  <chr>         <chr> <chr>                <chr>      <dbl>      <dbl>     <dbl>
1 Little Women  PG    Drama, Romance       Saoir~        91        7.8 108101214
2 Atonement     R     Drama, Mystery, Rom~ Keira~        85        7.8  50927067
3 The Conjuring R     Horror, Mystery, Th~ Patri~        68        7.5 137446368
4 Up            PG    Animation, Adventur~ Edwar~        88        8.3 293004164
5 Conclave      PG    Drama, Thriller      Ralph~        79        7.4  32580655
```

```
#  could use stringr functions to further organize this data - separate
#    different genres, different actors, etc.  But don't need to now.
```

On Your Own - Census

2. Adapt the code in `hennepin_tidycensus` to write a function called `MN_tract_data` to give the user choices about year, county, and variables to pull off. Show that `MN_tract_data(year = 2021, county = "Hennepin", variables = c("B01003_001", "B19013_001"))` works as expected. Make sure it also works for other years, counties, and variables (e.g. B25077_001 is median home price and B02001_002 is number of white residents).

```
MN_tract_data <- function(year, county, variables) {
  tidycensus::get_acs(
    year = year,
    state = "MN",
    geography = "tract",
    variables = variables,
    output = "wide",
    geometry = TRUE,
    county = county,
    show_call = TRUE
)
}
```

```
# MN_tract_data(year = 2021, county = "Hennepin", variables = c("B01003_001", "B19013_001"))
# MN_tract_data(year = 2018, county = "Dakota", variables = c("B25077_001", "B02001_002"))
# MN_tract_data(year = 2010, county = "Olmsted", variables = c("B25077_001", "B02001_002"))
```

3. Use your function from (2) along with `map` and `list_rbind` to build a data set for Rice county for the years 2019-2021. Use your scraped data to plot trends in income over time and population over time.

how do i get time on the data set?

```
MN_tract_data2 <- function(year) {
  tidycensus::get_acs(
    year = year,
    state = "MN",
    geography = "tract",
    variables = c("B01003_001E", "B19013_001E"),
    output = "wide",
    geometry = TRUE,
    county = "Rice",
    show_call = TRUE
)
}
rice_county <- map(2019:2021, MN_tract_data2) |>
  list_rbind() |>
  mutate(
    population = B01003_001E,
    median_income = B19013_001E
  )
```

```
# income over time
ggplot(rice_county, aes(x = median_income)) +
  geom_jitter()

# population over time
ggplot(rice_county, aes(x = population)) +
  geom_jitter()
```

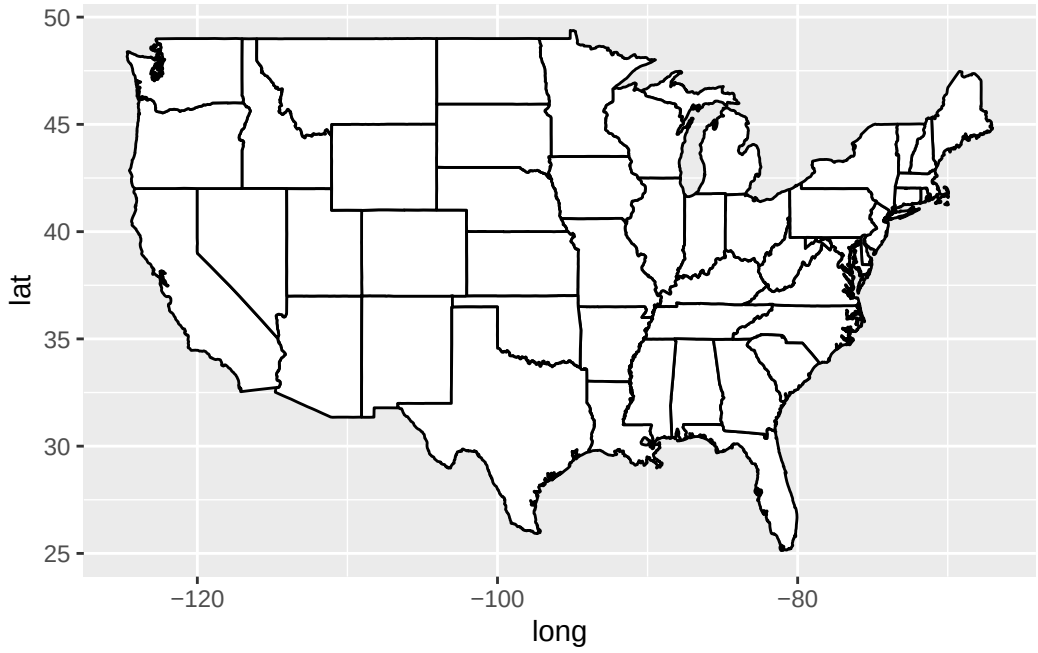
12_maps.qmd

```
# Initial packages required (we'll be adding more)
library(tidyverse)
library(mdsr)      # package associated with our MDSR book
```

```
library(maps)
us_states <- map_data("state")
head(us_states)
```

```
       long      lat group order  region subregion
1 -87.46201 30.38968     1     1 alabama      <NA>
2 -87.48493 30.37249     1     2 alabama      <NA>
3 -87.52503 30.37249     1     3 alabama      <NA>
4 -87.53076 30.33239     1     4 alabama      <NA>
5 -87.57087 30.32665     1     5 alabama      <NA>
6 -87.58806 30.32665     1     6 alabama      <NA>
```

```
us_states |>
  ggplot(mapping = aes(x = long, y = lat,
                       group = group)) +
  geom_polygon(fill = "white", color = "black")
```



[Pause to ponder:] What might the `group` and `order` columns represent?

Group may represent the state or regions. Order may be used to connect the coordinates in the correct order.

Other maps provided by the `maps` package include US counties, France, Italy, New Zealand, and two different views of the world. If you want maps of other countries or regions, you can often find them online.

```
vaccines <- read_csv("https://joeroith.github.io/264_spring_2026/Data/vacc_Mar21.csv")

vacc_mar13 <- vaccines |>
  filter(Date =="2021-03-13") |>
  select(State, Date, people_vaccinated_per100, share_doses_used, Governor)
vacc_mar13 <- vacc_mar13 |>
  mutate(State = str_to_lower(State))
```

oops, New York appears to be a problem.

```
vacc_mar13 |>
  anti_join(us_states, by = c("State" = "region"))

us_states |>
  anti_join(vacc_mar13, by = c("region" = "State")) |>
  count(region)
```

[Pause to ponder:] What did we learn by running `anti_join()` above?

New York in the vaccine data is labeled as "new york state" rather than just "new york" like in us_states.

You can still use data viz tools from Data Science 1 (like faceting) to create things like time trends in maps:

```
library(lubridate)
weekly_vacc <- vaccines |>
  mutate(State = str_to_lower(State)) |>
  mutate(State = str_replace(State, " state", ""),
         week = week(Date)) |>
  group_by(week, State) |>
  summarize(date = first(Date),
            mean_daily_vacc = mean(daily_vaccinated/est_population*1000)) |>
```

```
  right_join(us_states, by =c("State" = "region")) |>
  rename(region = State)

weekly_vacc |>
  filter(week > 2, week < 11) |>
  ggplot(mapping = aes(x = long, y = lat,
                       group = group)) +
  geom_polygon(aes(fill = mean_daily_vacc), color = "darkgrey",
               linewidth = 0.1) +
  labs(fill = "Weekly Average Daily Vaccinations per 1000") +
  coord_map() +
  theme_void() +
  scale_fill_viridis() +
  facet_wrap(~date) +
  theme(legend.position = "bottom")
```

[Pause to ponder:] are we bothered by the warning about many-to-many when you run the code above?

No, us_states has many states because each point for the coordinates gets its own row. There is also many states in the vaccines data since there is row for each state for each week.

```
library(statebins) # may need to install

vacc_mar13 |>
  mutate(State = str_to_title(State)) |>
  statebins(state_col = "State",
            value_col = "people_vaccinated_per100") +
  theme_statebins() +                                                    ①
  labs(fill = "People Vaccinated per 100")
```

① One nice layout. You can customize with usual ggplot themes.

[Pause to ponder:] Why might one use a map like above instead of our previous choropleth maps?

You can see the smaller states and their data better. No domination by larger states like Texas.

```
leaflet() |>
    addTiles() |>
    setView(lng = mean(airbnb.df$Long), lat = mean(airbnb.df$Lat),
            zoom = 13) |>
    addCircleMarkers(data = airbnb.df,
        lat = ~ Lat,
        lng = ~ Long,
        popup = ~ AboutListing,
        radius = ~ S_Accomodates,
        # These last options describe how the circles look
        weight = 2,
        color = "red",
        fillColor = "yellow")
```

[Pause to ponder:] List similarities and differences between leaflet plots and ggplots.

Similarities

- can add aesthetics (?) to determine what specific aspects of the plot look like (color, shape, etc.)
- similar syntax in leaflet's addCircleMarkers (or whatever add on) as in ggplot(data, aes())
  - assigning variables to different aspects

Differences

- leaflet is interactive while ggplot is static
- leaflet can use pipes while ggplot uses +

```
library(sf)                                                                    ①
states <- read_sf("https://rstudio.github.io/leaflet/json/us-states.geojson") ②
class(states)                                                                  ③
# states
```

```
[1] "sf"         "tbl_df"     "tbl"        "data.frame"
```

Now let's use `leaflet` to create an interactive plot!

```r
# Create our own category bins for population densities
#   and assign the yellow-orange-red color palette
bins <- c(0, 10, 20, 50, 100, 200, 500, 1000, Inf)
pal <- colorBin("YlOrRd", domain = states$density, bins = bins)

# Create labels that pop up when we hover over a state.  The labels must
#   be part of a list where each entry is tagged as HTML code.
library(htmltools)
library(glue)

states <- states |>
  mutate(labels = str_c(name, ": ", density, " people / sq mile"))

# If want more HTML formatting, use these lines instead of those above:
#states <- states |>
#  mutate(labels = glue("<strong>{name}</strong><br/>{density} people / #mi<sup>2</sup>"))

labels <- lapply(states$labels, HTML)

leaflet(states) |>
  setView(-96, 37.8, 4) |>
  addTiles() |>
  addPolygons(
    fillColor = ~pal(density),
    weight = 1,
    opacity = 1,
    color = "black",
    dashArray = "3",
    fillOpacity = 0.6,
    highlightOptions = highlightOptions(
      weight = 5,
      color = "red",
      dashArray = "7",
      fillOpacity = 1,
      bringToFront = TRUE),
    label = labels,
    labelOptions = labelOptions(
      style = list("font-weight" = "normal", padding = "3px 8px"),
      textsize = "15px",
      direction = "auto")) |>
  addLegend(pal = pal, values = ~density, opacity = 0.7, title = NULL,
    position = "bottomright")
```

[Pause to ponder:] Pick several formatting options in the code above, determine what they do, and then change them to create a customized look.

- dashArray: determines how close together the dashes are
- color: changes the color of the outline
- weight: determines the thickness of the line
- bringToFront: when TRUE, brings the additional line to the front of (overlapping) the previous line
- fillOpacity: changes the opacity of the state

On Your Own

1. Let's return to our example from above where we recreated an interactive choropleth map of population densities by US state. Recall how that plot was very suspicious? The population density data that came with the state geometries from our source seemed incorrect.

In `09_table_scraping.qmd` (On Your Own #1) we went out and found the real statewise population densities (from here) and created a tidy data frame. Now let's merge that with our state geometry shapefiles and regenerate our interactive choropleth map.

```
# Code from OYO #1 in 09_table_scraping.qmd

library(rvest)
library(polite)

session <- bow("https://en.wikipedia.org/wiki/List_of_states_and_territories_of_the_United_St

result <- scrape(session) |>
  html_nodes(css = "table") |>
  html_table(header = TRUE, fill = TRUE)
density_table <- result[[1]]
density_table

density_data <- density_table |>
  select(1, 2, 4, 5) |>
  filter(!row_number() == 1) |>
  rename(Land_area = `Land area`) |>
  mutate(state_name = str_to_lower(as.character(Location)),
         Density = parse_number(Density),
         Population = parse_number(Population),
         Land_area = parse_number(Land_area)) |>
```

```
  select(-Location)

density_data

# Merging
states2 <- read_sf("https://rstudio.github.io/leaflet/json/us-states.geojson")

states2 <- states2 |>
  select(name, geometry) |>
  mutate(name = str_to_lower(name)) |>
  rename(state_name = name)

states2 <- density_data |>
  right_join(states2)

states2 <- st_as_sf(states2)
class(states2)

# Plot
bins <- c(0, 10, 20, 50, 100, 200, 500, 1000, Inf)
density <- colorBin("YlOrRd", domain = states2$Density, bins = bins)

states2 <- states2 |>
  mutate(labels = str_c(state_name, ": ", Density, " people / sq mile"))
labels1 <- lapply(states2$labels, HTML)

leaflet(states2) |>
  setView(-96, 37.8, 4) |>
  addTiles() |>
  addPolygons(
    fillColor = ~density(Density),
    weight = 2,
    opacity = 1,
    color = "white",
    fillOpacity = 0.8,
    highlightOptions = highlightOptions(
      weight = 4,
      fillOpacity = 1,
      color = "black",
      bringToFront = TRUE),
    label = labels1,
    labelOptions = labelOptions(
```

```
      style = list("font-weight" = "normal", padding = "3px 8px"),
      textsize = "15px",
      direction = "auto")) |>
  addLegend(pal = density, values = ~Density, opacity = 0.7, title = NULL,
    position = "bottomright")
  )
```

2. The `states` dataset in the `poliscidata` package contains 135 variables on each of the 50 US states. See here for more detail.

Your task is to create a *two* meaningful choropleth plots, one using a numeric variable and one using a categorical variable from `poliscidata::states`. You should make *two* versions of each plot: a static plot using the `maps` package and `ggplot()`, and an interactive plot using the `sf` package and `leaflet()`. Write a sentence or two describing what you can learn from each plot.

Here's some R code and hints to get you going:

```
# Get info to draw US states for geom_polygon (connect the lat-long points)
library(maps)
states_polygon <- as_tibble(map_data("state")) |>
  select(region, group, order, lat, long)

# See what the state (region) levels look like in states_polygon

#   unique(states_polygon$region)


# Get info to draw US states for geom_sf and leaflet (simple features object
#   with multipolygon geometry column)
library(sf)
states_sf <-
  read_sf("https://rstudio.github.io/leaflet/json/us-states.geojson") |>
  select(name, geometry)

# See what the state (name) levels look like in states_sf

#   unique(states_sf$name)


# Load in state-wise data for filling our choropleth maps
#   (Note that I selected my two variables of interest to simplify)
library(poliscidata)   # may have to install first
```

```r
polisci_data <- as_tibble(poliscidata::states) |>
  select(state, carfatal07, cook_index3)

# See what the state (state) levels look like in polisci_data

#   unique(polisci_data$state)   # can't see trailing spaces but can see
                                 #   lack of internal spaces
#   print(polisci_data)   # can see trailing spaces
```

R code hints:

- stringr functions like `str_squish` and `str_to_lower` and `str_replace_all` (be sure to carefully look at your keys!)
- *_join functions (make sure they preserve classes)
- filter so that you only have 48 contiguous states (and maybe DC)
- for help with colors: https://rstudio.github.io/leaflet/reference/colorNumeric.html
- be sure labels pop up when scrolling with leaflet

```r
# Make sure all keys have the same format before joining:
#   all lower case, no internal or external spaces

# states_polygon
states_polygon <- states_polygon |>
  rename(state = region) |>
  filter(!state %in% c("district of columbia")) |>
  mutate(state = str_replace_all(state, " ", ""))

# state_sf
states_sf <- states_sf |>
  mutate(name = str_replace_all(name, " ", "")) |>
  rename(state = name)

states_sf <- states_sf |>
  filter(!state %in% c("Alaska", "Hawaii", "DistrictofColumbia", "PuertoRico")) |>
  mutate(state = str_to_lower(state))

# polisci
polisci_data <- polisci_data |>
  mutate(state = str_squish(state)) |>
  filter(!state %in% c("Alaska", "Hawaii")) |>
  mutate(state = str_to_lower(state))
```

```
# Now we can merge data sets together for the static and the interactive plots


# Merge with states_polygon (static)
gg_states <- polisci_data |>
  full_join(states_polygon)
```

```
Joining with `by = join_by(state)`
```

```
# Check that merge worked for 48 contiguous states
#   unique(gg_states$state)

# Merge with states_sf (static or interactive)
int_states <- polisci_data |>
  full_join(states_sf)
```

```
Joining with `by = join_by(state)`
```

```
# Check that merge worked for 48 contiguous states
#   unique(int_states$state)
int_states <- st_as_sf(int_states)
class(int_states)
```

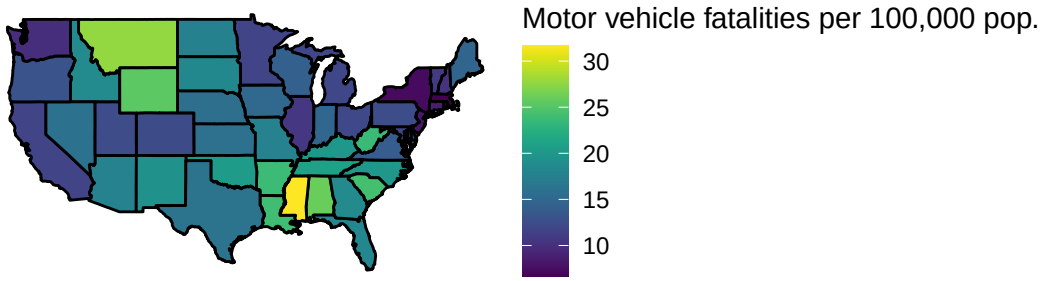
```
[1] "sf"         "tbl_df"     "tbl"        "data.frame"
```

Numeric variable (static plot): Here, you can see that Mississippi and Montana had the most car fatalities per 100,000 in 2007 as they are the lightest in color. The Northeast region of the U.S. seems to have the least amount of car fatalities per 100,000 as they are the darkest states.

```
library(viridis)
```

```
Loading required package: viridisLite


Attaching package: 'viridis'

The following object is masked from 'package:maps':

    unemp
```

```
gg_states |>
  ggplot(aes(x = long, y = lat, group = group)) +
  geom_polygon(aes(fill = carfatal07), color = "black") +
  coord_map() +
  scale_fill_viridis() +
  theme_void() +
  labs(fill = "Motor vehicle fatalities per 100,000 pop.")
```



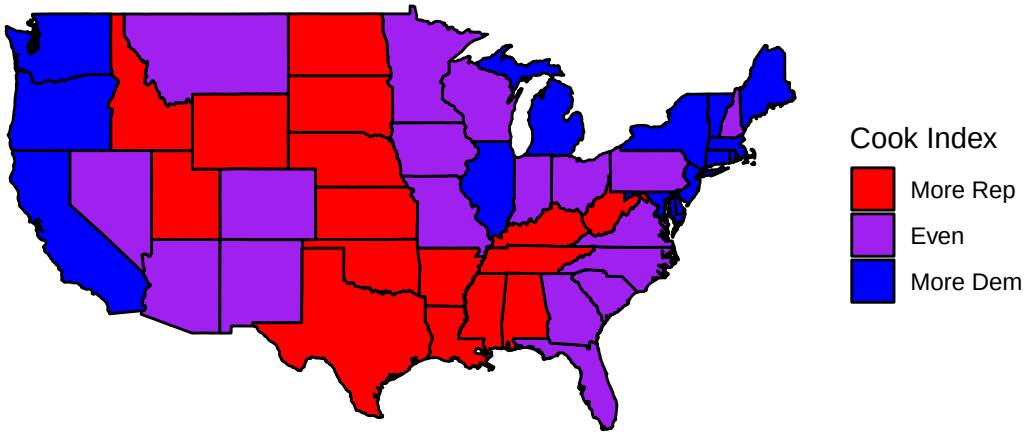
Numeric variable (interactive plot): Here, you can see exactly how many average car fatalities occur in each state instead of inferring from the color and legend.

```
# it's okay to skip a legend here

library(leaflet)
library(htmltools)
library(glue)

bins <- c(0, 5, 10, 15, 20, 25, 30, 35)
pal <- colorBin("YlOrRd", domain = int_states$carfatal07, bins = bins)
int_states <- int_states |>
  mutate(labels = str_c(str_to_title(state), ": ", carfatal07, " motor vehicle fatalities / 
labels <- lapply(int_states$labels, HTML)
```

```r
leaflet(int_states) |>
  setView(-96, 37.8, 4) |>
  addTiles() |>
  addPolygons(
    fillColor = ~pal(carfatal07),
    weight = 2,
    opacity = 1,
    color = "black",
    dashArray = "3",
    fillOpacity = 1,
    highlightOptions = highlightOptions(
      weight = 5,
      color = "red",
      dashArray = "",
      fillOpacity = 0.7,
      bringToFront = TRUE),
    label = labels,
    labelOptions = labelOptions(
      style = list("font-weight" = "normal", padding = "3px 8px"),
      textsize = "15px",
      direction = "auto")) |>
  addLegend(pal = pal, values = ~carfatal07, opacity = 0.7, title = NULL,
    position = "bottomright")
  )
```

Categorical variable (static plot): In this plot, you can generally see where states and regions of the U.S. lean politically.

```r
# be really careful with matching color order to factor level order
gg_states |>
  ggplot(aes(x = long, y = lat, group = group)) +
  geom_polygon(aes(fill = cook_index3), color = "black") +
  coord_map() +
  scale_fill_manual(values = c("red", "purple", "blue")) +
  theme_void() +
  labs(fill = "Cook Index")
```



Categorical variable (interactive plot): With the interactive version, you can zoom in/hover on the smaller states to see where they lean politically more easily than you can on the static map.

```
# may use colorFactor() here
colors <- colorFactor(c("red", "purple", "blue"), domain = int_states$cook_index3)

int_states <- int_states |>
  mutate(labels2 = str_c(str_to_title(state), " : ", cook_index3))
labels2 <- lapply(int_states$labels2, HTML)

leaflet(int_states) |>
  setView(-96, 37.8, 4) |>
  addTiles() |>
  addPolygons(
    fillColor = ~colors(cook_index3),
    weight = 2,
    opacity = 1,
    color = "white",
    fillOpacity = 0.7,
    highlightOptions = highlightOptions(
      weight = 5,
      color = "black",
```

```
    bringToFront = TRUE),
  label = labels2,
  labelOptions = labelOptions(
    style = list("font-weight" = "normal", padding = "3px 8px"),
    textsize = "15px",
    direction = "auto")) |>
addLegend(pal = colors, values = ~cook_index3, opacity = 0.7, title = NULL,
  position = "bottomright")
  )
)
```